

O&G · 3 — Universal Differential Equations: material balance with unknown aquifer support ⇌

A reservoir tank material-balance model tracks average reservoir pressure P as fluid is withdrawn:

$$\frac{dP}{dt} = -c_1 q(t) + (\text{pressure support})$$

The depletion term $-c_1 q(t)$ (rate times a known compressibility/pore-volume factor) is usually well understood. What's often *not* well characterized is aquifer influx or injection support — its strength and functional form are exactly the kind of thing reservoir engineers argue about. We keep the known depletion term as an equation and let a neural network learn the unknown support term directly from the pressure history, following the Universal Differential Equations recipe of `03_universal_differential_equations.jl`.

```
1 using Lux, OrdinaryDiffEq, SciMLSensitivity, Optimization, OptimizationOptimisers,  
   ComponentArrays, Random, Plots
```

1. Ground truth: depletion plus a Fetkovich-style aquifer ⇌

We generate synthetic pressure data from a full model that *does* include an aquifer influx term $J(P_i - P)$ (a common approximation for a weak, tank-like aquifer) — but this term is what we'll later hide from the identified model.

```
production_rate (generic function with 1 method)
```

```
1 production_rate(t) = t < 2.0 ? 500.0 : 800.0 # a second well comes online at t = 2 yr
```

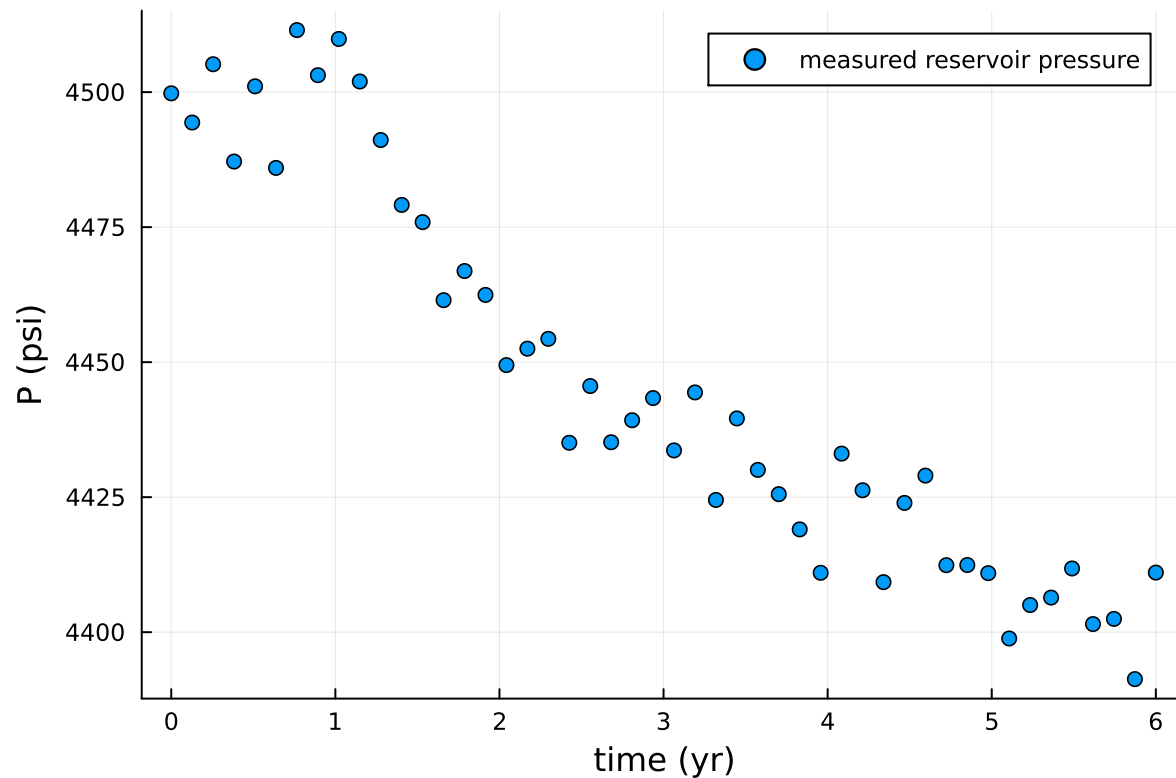
```
tank_true! (generic function with 1 method)
```

```
1 function tank_true!(du, u, p, t)  
2     P = u[1]  
3     c1, J, Pi = p  
4     q = production_rate(t)  
5     du[1] = -c1 * q + J * (Pi - P)  
6     return nothing  
7 end
```

```
1x48 Matrix{Float64}:
```

```
4499.78 4494.37 4505.17 4487.15 4501.08 ... 4401.5 4402.46 4391.31 4411.04
```

```
1 begin
2   rng_og3 = Random.default_rng()
3   Random.seed!(rng_og3, 21)
4
5   Pi_true = 4500.0           # psi, initial reservoir pressure
6   c1_true = 0.02
7   J_true = 0.015
8   u0_tank = [Pi_true]
9   tspan_tank = (0.0, 6.0)
10  tsteps_tank = range(tspan_tank[1], tspan_tank[2]; length=48)
11
12  tank_prob = ODEProblem(tank_true!, u0_tank, tspan_tank, (c1_true, J_true, Pi_true))
13  tank_sol = solve(tank_prob, Tsit5(); saveat=tsteps_tank, tstops=[2.0])
14  pressure_data = Array(tank_sol) .+ 8.0 .* randn(rng_og3, 1, length(tsteps_tank))
15 end
```



```
1 scatter(tsteps_tank, pressure_data[1, :]; label="measured reservoir pressure",
2         xlabel="time (yr)", ylabel="P (psi)")
```

2. The hybrid model ⇔

We keep the depletion term $-c_1 q(t)$ exactly (assume c_1 is known from PVT and pore-volume data), and replace the aquifer-support term with a small neural network $U_\theta(P)$.

```
ComponentVector{Float32}(layer_1 = (weight = Float32[-2.7910485; -2.0032663; ... ; 2.127652; -
```

```
1 begin
2   support_nn = Chain(Dense(1, 8, tanh), Dense(8, 8, tanh), Dense(8, 1))
3   support_ps_init, support_st = Lux.setup(rng_og3, support_nn)
4   support_ps_init = ComponentArray(support_ps_init)
5 end
```

```
c1_known = 0.02
```

```
1 c1_known = 0.02
```

```
tank_ude! (generic function with 1 method)
```

```
1 function tank_ude!(du, u, p, t)
2   P = u[1]
3   q = production_rate(t)
4   support, _ = support_nn([P / 1000.0], p, support_st)
5   du[1] = -c1_known * q + 1000.0 * support[1]
6   return nothing
7 end
```

```
tank_ude_prob = ODEProblem with uType Vector{Float64} and tType Float64. In-place: true
Non-trivial mass matrix: false
timespan: (0.0, 6.0)
u0: 1-element Vector{Float64}:
4500.0
```

```
1 tank_ude_prob = ODEProblem(tank_ude!, u0_tank, tspan_tank)
```

3. Train the neural support term against the pressure history ⇔

```
predict_tank (generic function with 1 method)
```

```
1 function predict_tank(p)
2   solve(tank_ude_prob, Tsit5(); p=p, saveat=tsteps_tank, tstops=[2.0],
3     sensealg=InterpolatingAdjoint(; autojacvec=ZygoteVJP())) |> Array
4 end
```

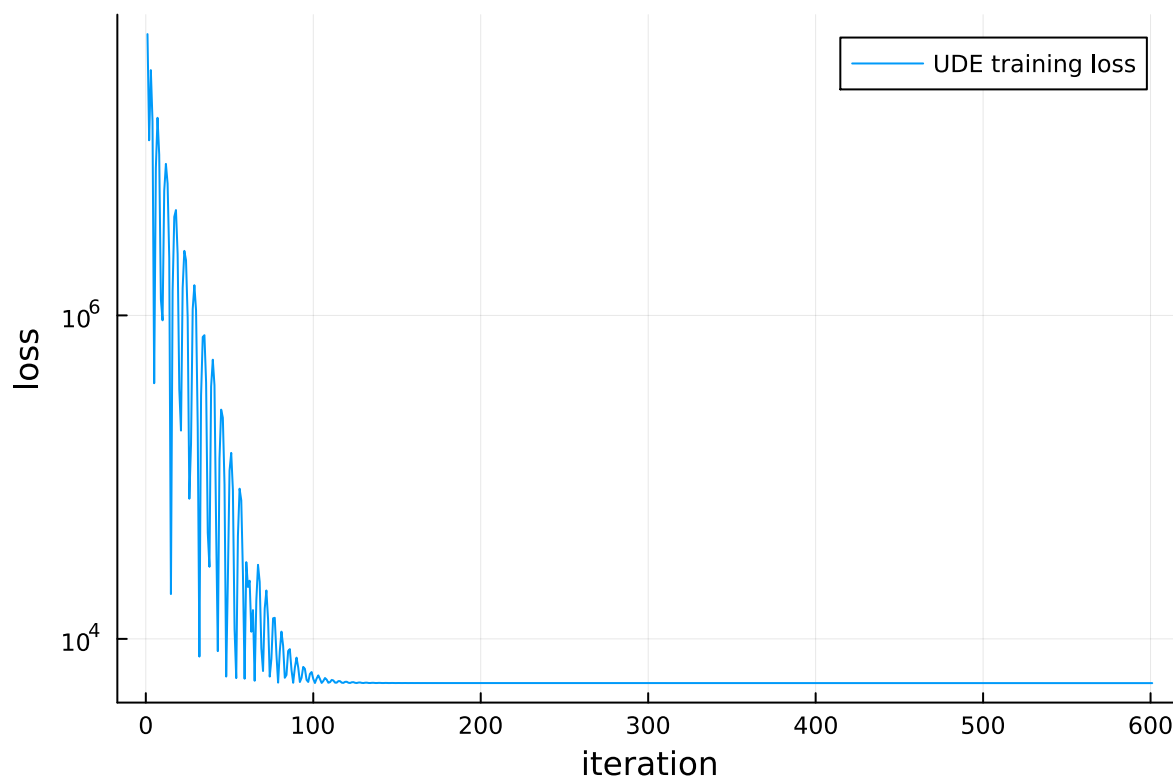
```
loss_tank (generic function with 1 method)
```

```
1 function loss_tank(p, _)
2   pred = predict_tank(p)
3   size(pred) == size(pressure_data) || return Inf
4   return sum(abs2, pressure_data .- pred)
5 end
```

► SciMLBase.OptimizationSolution{Float32, 1, ComponentArrays.ComponentVector{Float32, Vector{Float32}}

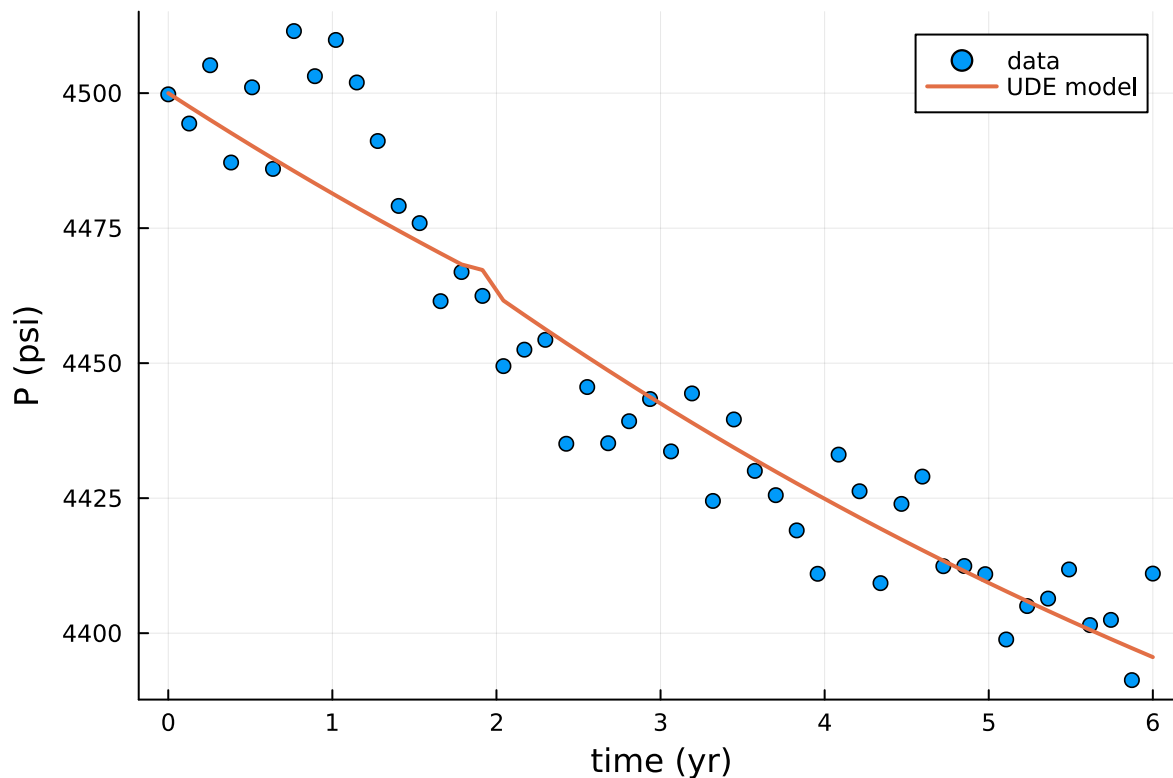
```
1 begin
2     optf_og3 = OptimizationFunction(loss_tank, Optimization.AutoZygote())
3     optprob_og3 = OptimizationProblem(optf_og3, support_ps_init)
4
5     tank_losses = Float64[]
6     tank_callback = function (state, loss)
7         push!(tank_losses, loss)
8         return false
9     end
10
11     res_tank = Optimization.solve(optprob_og3, OptimizationOptimisers.Adam(0.03);
12     callback=tank_callback, maxiters=600)
13 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.



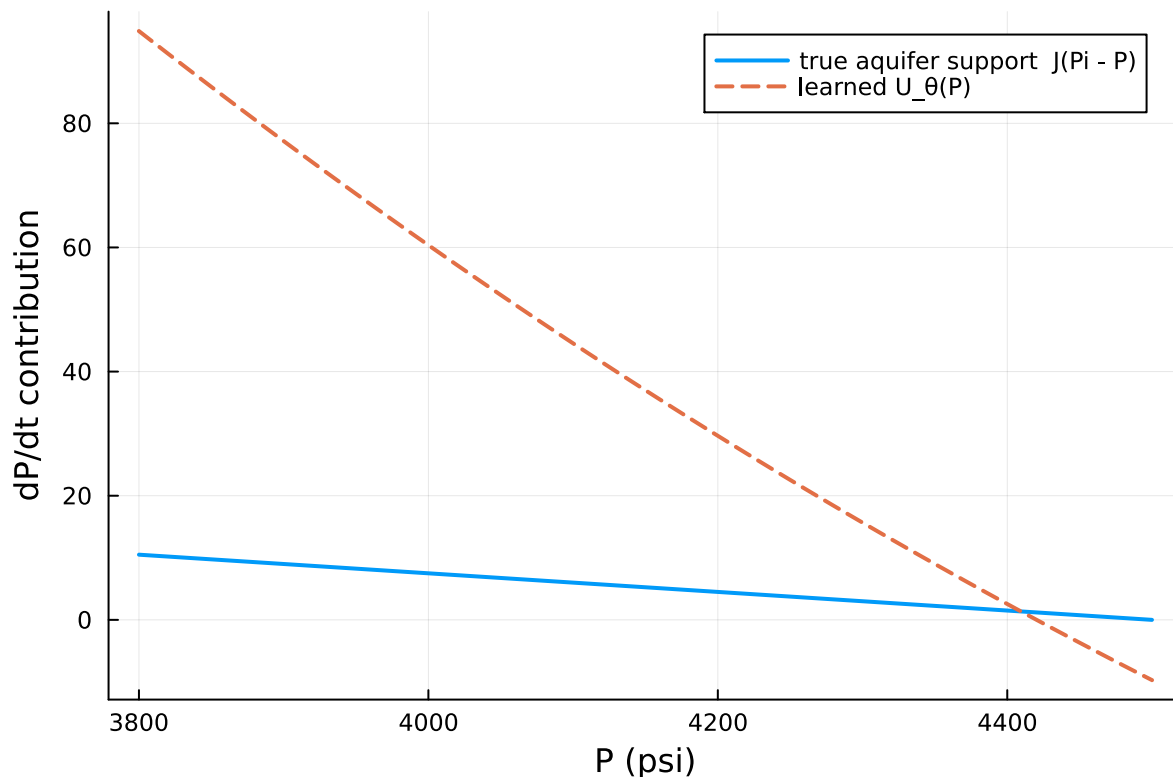
```
1 plot(tank_losses; xlabel="iteration", ylabel="loss", yscale=:log10, label="UDE training loss")
```

4. Fit quality and the recovered support curve ⇌



```
1 begin
2     tank_fit = predict_tank(res_tank.u)
3     scatter(tsteps_tank, pressure_data[1, :]; label="data", xlabel="time (yr)",
4             ylabel="P (psi)")
5     plot!(tsteps_tank, tank_fit[1, :]; label="UDE model", lw=2)
6 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.



```

1 begin
2     p_grid = 3800.0:10.0:4500.0
3     learned_support = [1000.0 * first(support_nn([p / 1000.0], res_tank.u, support_st)
4                          [1]) for p in p_grid]
5     true_support = [J_true * (Pi_true - p) for p in p_grid]
6     plot(p_grid, true_support; label="true aquifer support J(Pi - P)", lw=2, xlabel="P
7           (psi)", ylabel="dP/dt contribution")
8     plot!(p_grid, learned_support; label="learned U_theta(P)", lw=2, ls=:dash)
9 end

```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] × B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.

Takeaways ⇄

- Only the aquifer term — the part reservoir engineers are genuinely uncertain about — was replaced by a network; the depletion physics stayed a normal equation.
- Because `production_rate(t)` has a jump at $t = 2$, passing `tstops=[2.0]` to `solve` keeps the adaptive integrator accurate across the discontinuity instead of stepping over it.
- The recovered $U_\theta(P)$ curve is itself useful: comparing its shape to the true influx law is exactly how an engineer would sanity-check whether the identified support behaves like a real aquifer (roughly linear in drawdown) or something else entirely (e.g. injection support, which would look different).

